First-Order Logic

Chapter 8

Outline

- Why FOL?
- Syntax and semantics of FOL
- Using FOL
- Wumpus world in FOL
- Knowledge engineering in FOL

Pros and cons of propositional logic

☺ Propositional logic is declarative

☺ Propositional logic allows partial/disjunctive/negated information
- (unlike most data structures and databases)

☺ Propositional logic is compositional:
- meaning of $B_{1,1} \wedge P_{1,2}$ is derived from meaning of $B_{1,1}$ and of $P_{1,2}$

☺ Meaning in propositional logic is context-independent
- (unlike natural language, where meaning depends on context)

☹ Propositional logic has very limited expressive power
- (unlike natural language)
- E.g., cannot say "pits cause breezes in adjacent squares“
  - except by writing one sentence for each square

First-order logic

- Whereas propositional logic assumes the world contains facts,
- first-order logic (like natural language) assumes the world contains
- Objects: people, houses, numbers, colors, baseball games, wars, …
- Relations: red, round, prime, brother of, bigger than, part of, comes between, …
- Functions: father of, best friend, one more than, plus, …

First-order logic

- FOPL is also called predicate calculus.
- It is an AI language representation that has:
  - Well-defined formal semantics and
  - Sound and complete inference rules.

Features of FOPL:-

- It offers a formal approach to reasoning with a sound theoretical foundation.
- It provides flexibility in representing natural language.
- It is widely accepted by the workers in AI field as one of the most useful representation methods.

Approaches of FOPL

- “ symbolic structures”
  - Symbolic structures are built to represent natural language statements using predicates, functions, variables, constants, quantifiers and logical connectives.
- “inference rules”
  - Inference rules are applied on the above structures to deduce new structures for interpretation.

Example:-

" All employee of INFOSYS are software engineers."

$\forall x$ (INFOSYS(X) $\Rightarrow$ SW-ENGG(X))

"Ravi is an employee of INFOSYS "

INFOSYS(Ravi)

So it concludes:-

"Ravi is a software engineer"

SW-ENGG(Ravi)

Syntax of FOL: Basic elements

- Connectives $\neg, \Rightarrow, \wedge, \vee, \Leftrightarrow$
- Equality =
- Constants : they are fixed value terms, belong to a given domain.

  Example:- KingJohn, 2, NUS,...
- Quantifiers $\forall$ (universal), $\exists$ (existential)
- Auxiliary symbols : like),(,[],{} are used for punctuation.

Syntax of FOL: Basic elements

- Variables : they are terms that can assume different values over a given domain. It is denoted by letters x, y, a, b,...
- Functions : function symbols defined over a domain map n elements (n>0) to a single element of the domain. Here n is called the rank or degree of a function. Examples :- Sqrt, LeftLegOf,...

Syntax of FOL: Basic elements

- terms : constant variables and functions are called terms.
- Predicates : they denote relations or functional mapping from the elements of a domain to the values true or false. For example :- Brother, >,... . Like functions, predicates can have n (n>= 0) terms as argument. A 0-ary predicate is a proposition. i.e. propositions are constant predicates.

Atomic sentences

Predicates are called atomic sentences or atoms.

Atomic sentence = *predicate* (*term*$_1$,...,*term*$_n$)
or *term*$_1$ = *term*$_2$

Term = *function* (*term*$_1$,...,*term*$_n$)
or *constant* or *variable*

- E.g., *Brother(KingJohn, RichardTheLionheart)*
  >(Length(LeftLegOf(Richard))
  Length(LeftLegOf(KingJohn)))

Complex sentences

- Complex sentences are made from atomic sentences using connectives

$$\neg S,\ S_1 \wedge S_2,\ S_1 \vee S_2,\ S_1 \Rightarrow S_2,\ S_1 \Leftrightarrow S_2,$$

E.g. *Sibling(KingJohn,Richard)* $\Rightarrow$ *Sibling(Richard,KingJohn)*

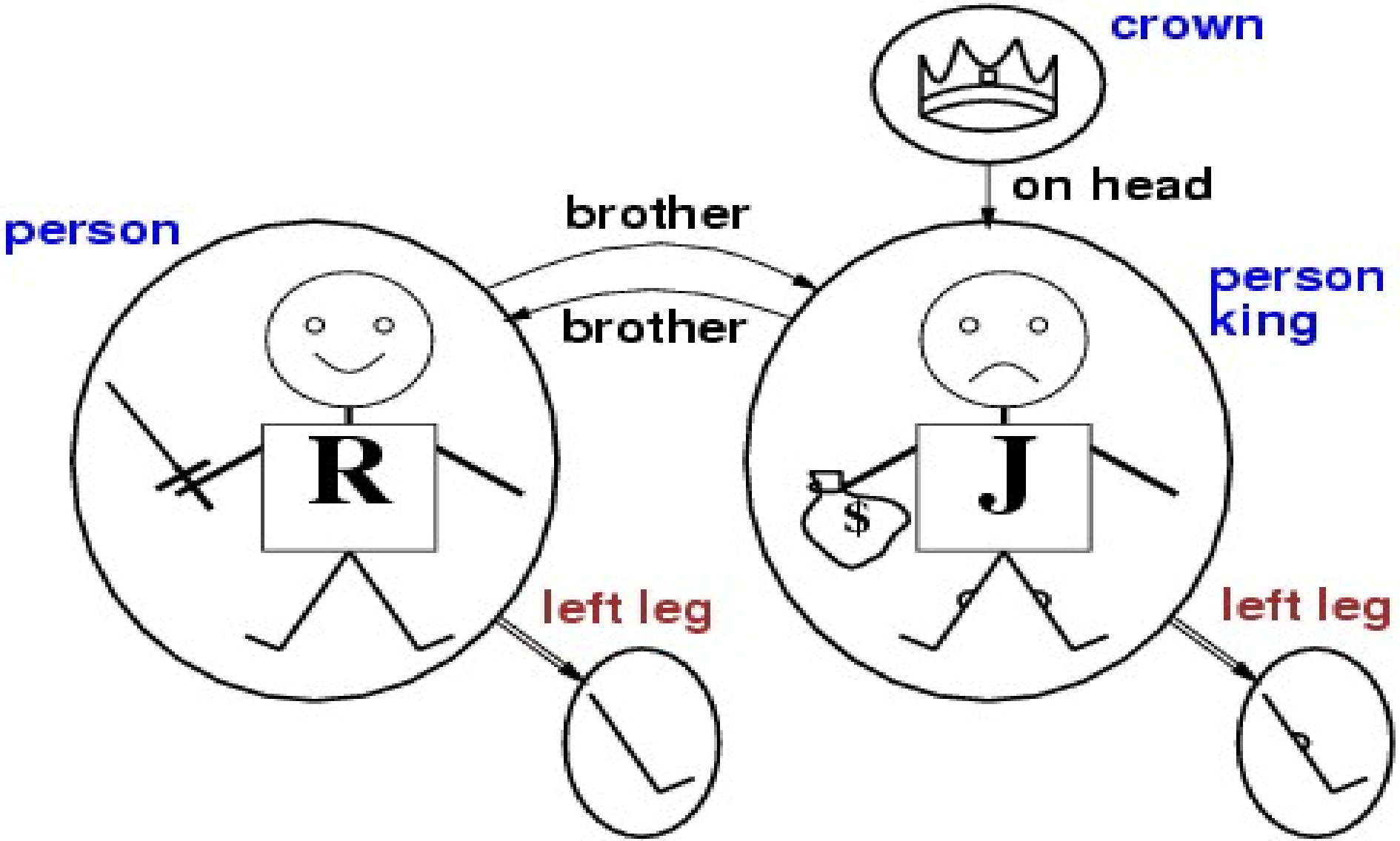
$>(1,2) \vee \leq (1,2)$

$>(1,2) \wedge \neg >(1,2)$

Truth in first-order logic

- Sentences are true with respect to a model and an interpretation
- Model contains objects (domain elements) and relations among them
- Interpretation specifies referents for
  constant symbols → objects
  predicate symbols → relations
  function symbols → functional relations
- An atomic sentence *predicate*($term_1$,...,$term_n$) is true iff the objects referred to by $term_1$,...,$term_n$ are in the relation referred to by *predicate*

Models for FOL: Example



Universal quantification

- $\forall$<*variables*> <*sentence*>

  Everyone at NUS is smart:

  $\forall x\ At(x,NUS) \Rightarrow Smart(x)$

$\forall x$ *P* is true in a model *m* iff *P* is true with *x* being each possible object in the model

- Roughly speaking, equivalent to the conjunction of instantiations of *P*

  $At(KingJohn,NUS) \Rightarrow Smart(KingJohn)$
  $\wedge\ At(Richard,NUS) \Rightarrow Smart(Richard)$
  $\wedge\ At(NUS,NUS) \Rightarrow Smart(NUS)$
  $\wedge$ ...

A common mistake to avoid

- Typically, $\Rightarrow$ is the main connective with $\forall$
- Common mistake: using $\wedge$ as the main connective with $\forall$:

  $\forall x$ At(x,NUS) $\wedge$ Smart(x)

  means “Everyone is at NUS and everyone is smart”

Existential quantification

- ∃<*variables*> <*sentence*>

- Someone at NUS is smart:
- ∃*x* At(x,NUS) ∧ Smart(x)

- ∃*x* *P* is true in a model *m* iff *P* is true with *x* being some possible object in the model

- Roughly speaking, equivalent to the disjunction of instantiations of *P*
- At(KingJohn,NUS) ∧ Smart(KingJohn)
  ∨ At(Richard,NUS) ∧ Smart(Richard)
  ∨ At(NUS,NUS) ∧ Smart(NUS)
  ∨ ...

Another common mistake to avoid

- Typically, $\wedge$ is the main connective with $\exists$
- Common mistake: using $\Rightarrow$ as the main connective with $\exists$:
- $\exists x$ At(x,NUS) $\Rightarrow$ Smart(x)

  is true if there is anyone who is not at NUS!

Properties of quantifiers

- $\forall x\ \forall y$ is the same as $\forall y\ \forall x$
- $\exists x\ \exists y$ is the same as $\exists y\ \exists x$
- $\exists x\ \forall y$ is not the same as $\forall y\ \exists x$
- $\exists x\ \forall y$ Loves(x,y)
  - "There is a person who loves everyone in the world"
- $\forall y\ \exists x$ Loves(x,y)
  - "Everyone in the world is loved by at least one person"

- Quantifier duality: each can be expressed using the other
-
- $\forall x$ Likes(x,IceCream) $\quad\quad \neg\exists x\ \neg$Likes(x,IceCream)
-
- $\exists x$ Likes(x,Broccoli) $\quad\quad \neg\forall x\ \neg$Likes(x,Broccoli)
-

Properties of quantifiers…

- *Equality:* $term_1 = term_2$ is true under a given interpretation if and only if $term_1$ and $term_2$ refer to the same object
- E.g., Father(John) = Henry

The kinship domain (the domain of family relationships) :

definition of *Sibling* in terms of *Parent*:

$$\forall x,y\ Sibling(x,y) \Leftrightarrow [\neg(x = y) \land \exists m,f \neg (m = f) \land Parent(m,x) \land Parent(f,x) \land Parent(m,y) \land Parent(f,y)]$$

Using FOL

Brothers are siblings

$\forall x,y\ Brother(x,y) \Leftrightarrow Sibling(x,y)$

One's mother is one's female parent

$\forall m,c\ Mother(c) = m \Leftrightarrow (Female(m) \wedge Parent(m,c))$

“Sibling” is symmetric

$\forall x,y\ Sibling(x,y) \Leftrightarrow Sibling(y,x)$

Using FOL

The set domain:

- $\forall s\ Set(s) \Leftrightarrow (s = \{\}) \vee (\exists x, s_2\ Set(s_2) \wedge s = \{x|s_2\})$
- $\neg\exists x, s\ \{x|s\} = \{\}$
- $\forall x, s\ x \in s \Leftrightarrow s = \{x|s\}$
- $\forall x, s\ x \in s \Leftrightarrow [\exists y, s_2\} (s = \{y|s_2\} \wedge (x = y \vee x \in s_2))]$
- $\forall s_1, s_2\ s_1 \subseteq s_2 \Leftrightarrow (\forall x\ x \in s_1 \Rightarrow x \in s_2)$
- $\forall s_1, s_2\ (s_1 = s_2) \Leftrightarrow (s_1 \subseteq s_2 \wedge s_2 \subseteq s_1)$
- $\forall x, s_1, s_2\ x \in (s_1 \cap s_2) \Leftrightarrow (x \in s_1 \wedge x \in s_2)$
- $\forall x, s_1, s_2\ x \in (s_1 \cup s_2) \Leftrightarrow (x \in s_1 \vee x \in s_2)$

Example :-

- Marcus was a man.

  man(Marcus)
- Marcus was a Pompeian.

  pompeian(Marcus)
- All Pompeians were Romans.

  $\forall x$ [Pompeian(x) $\Rightarrow$ Roman(x)]
- Caesar was a ruler.

  ruler(Caesar)
- All Romans were either loyal to Caesar or hated him.

  $\forall x$ [Roman(x) $\Rightarrow$ loyalto(x,Caesar) $\vee$ hate(x, Caesar)]

Example :-

- Everyone is loyal to some one.

  $\forall x [\exists y \ \text{loyalto}(x,y)]$

- people only try to assassinate rulers they are not loyal to.

  $\forall x \forall y [\text{person}(x) \wedge \text{ruler}(y) \wedge \text{tryassassinate}(x,y) \Rightarrow \sim \text{loyalto}(x,y)]$

- Marcus tried to assassinate Caesar.

  tryassassinate(Marcus,Caesar)

  we can note that we are ignoring the issue of tense.

- Every person likes the roses.
- All green snakes are poisonous.
- “Every house is a physical object” is translated as
- ∀ x:(house(x) -> physical object(x));
- “Some physical objects are houses”

$$\exists x.(\text{physical\_object}(x) \wedge \text{house}(x))$$

- ”Every house has an owner” or, equivalently, “every house is owned by somebody” is translated as

$$\forall x(\textbf{house}(x) \rightarrow \exists y.\textbf{owns}(y, x)),$$

"Everybody owns a house" is translated as

$$\forall x.\exists y.(\textbf{owns}(x, y) \wedge \textbf{house}(y))$$

- "Sue owns a house" is translated as

$$\exists x.(\mathbf{owns}(\mathbf{Sue}, x) \wedge \mathbf{house}(x))$$

  where **Sue** is an individual constant symbol.

- "Peter does not own a house" is translated as

$$\neg\exists x.(\mathbf{owns}(\mathbf{Peter}, x) \wedge \mathbf{house}(x))$$

- "Somebody does not own a house" is translated as

$$\exists x.\forall y.(\mathbf{owns}(x, y) \rightarrow \neg\mathbf{house}(y))$$

- there are houses: $\exists x.\mathbf{house}(x)$
- there are human beings: $\exists x.\mathbf{human}(x)$
- no house is a human being: $\forall x.(\mathbf{house}(x) \rightarrow \neg\mathbf{human}(x))$
- some humans own a house: $\exists x.\exists y.(\mathbf{human}(x) \wedge \mathbf{house}(y) \wedge \mathbf{owns}(x, y))$
- Sue is human: $\mathbf{human}(\mathbf{Sue})$
- Sue does not own a house: $\neg\exists x.(\mathbf{owns}(\mathbf{Sue}, x) \wedge \mathbf{house}(x))$
- every house has an owner: $\forall x.(\mathbf{house}(x) \rightarrow \exists y.\mathbf{owns}(y, x))$

Interacting with FOL KBs

- Suppose a wumpus-world agent is using an FOL KB and perceives a smell and a breeze (but no glitter) at *t=5*:

  `Tell`(KB,Percept([Smell,Breeze,None],5))
  `Ask`(KB,∃a BestAction(a,5))

- I.e., does the KB entail some best action at *t=5*?
- Answer: *Yes*, {*a/Shoot*} ← substitution (binding list)

- Given a sentence *S* and a substitution σ,
- *S*σ denotes the result of plugging σ into *S*; e.g.,
  S = Smarter(x,y)
  σ = {x/Hillary,y/Bill}
  *S*σ = Smarter(Hillary,Bill)
- `Ask`(KB,S) returns some/all σ such that KB ⊨ σ

Knowledge base for the wumpus world

- Perception
  - $\forall t,s,b$ Percept([s,b,Glitter],t) $\Rightarrow$ Glitter(t)
  - –
- Reflex
  - $\forall t$ Glitter(t) $\Rightarrow$ BestAction(Grab,t)

Deducing hidden properties

- $\forall$x,y,a,b *Adjacent*([x,y],[a,b]) $\Leftrightarrow$
  [a,b] $\in$ {[x+1,y], [x-1,y],[x,y+1],[x,y-1]}

Properties of squares:

- $\forall$s,t *At*(Agent,s,t) $\wedge$ Breeze(t) $\Rightarrow$ Breezy(s)
- Squares are breezy near a pit:
  - Diagnostic rule---infer cause from effect

    $\forall$s Breezy(s) $\Rightarrow$ \Exi{r} Adjacent(r,s) $\wedge$ Pit(r)$

  - Causal rule---infer effect from cause

    $\forall$r Pit(r) $\Rightarrow$ [$\forall$s Adjacent(r,s) $\Rightarrow$ Breezy(s)$]

Knowledge engineering in FOL

1. Identify the task
2. Assemble the relevant knowledge
3. Decide on a vocabulary of predicates, functions, and constants
4. Encode general knowledge about the domain
5. Encode a description of the specific problem instance
6. Pose queries to the inference procedure and get answers
7. Debug the knowledge base

Summary

- First-order logic:
-
  - objects and relations are semantic primitives
  - syntax: constants, functions, predicates, equality, quantifiers
- Increased expressive power: sufficient to define wumpus world
-